FoodBridge Navigator

Lab 3: AI-based project initialisation

Prepared 18 October 2026 | Technology stack and project structure | Sections 3.3.1-3.3.5

Method and inputs

This report evaluates the FoodBridge technology stack and project structure using the updated SRS, class relationships checked against the existing implementation, and the earlier Lab 2 stack exercise [1]. The accompanying TODO-only template represents these responsibilities separately from the full application.

The prompts below are reusable specifications for this review, and the responses set out the AI's assessment. They are not a transcript of a new, independent fresh-chat experiment. The earlier model's identity, settings and complete diagram revision are unavailable, so the comparison focuses on the recorded recommendations and updated design responsibilities.

3.3.1 - Updated input supplied to the exercise

System description: FoodBridge helps registered users in Singapore find approved food support, check suitability, freshness and eligibility, plan routes, contact providers, save services and report errors. Administrators use a separate browser portal to review reports, approve changes and inspect audits. Bookmarks and optional daily reminders are local and account-separated. Hours, scheduled sessions and food availability are different facts. Device origins and routes remain transient. Imports require review and provenance; unknown information remains explicit.

Updated class input: use the diagram on page 3 and its source map. MainActivity observes AppViewModel/UiState; AppViewModel uses Repository and ReminderStore; SessionStore protects local credentials. Backend Service inherits Origin and contains Schedule/Observation; Search inherits Origin. Record stores typed JSON and versions. Account has AccountSession/AccountLink children. Audit records logical actor/target references. Provider is distinct from a service location. Reports and changes are JSON-backed records. ReminderReceiver dispatches local reminder work.

Text prompt A - Repeat the recorded Lab 2 prompt

Based on the supplied FoodBridge Navigator system description and Lab 2 class diagrams, recommend a technology stack appropriate for an entry-level software engineering team. The first release targets Android 12+ in Singapore. It needs an Android UI, a small API for approved service search and moderation, local bookmarks/preferences, a persistent server-side data store, and OneMap integration. Keep the proposal small and explain the role of each technology. Treat the proposal as a discussion starting point.

The prompt wording is retained from Lab 2 [1]. The input context is the updated description and class model, with Kotlin, Python and plain JavaScript as the preferred technologies. Changes in scope and input context are considered when comparing the recommendations.

3.3.1 - AI response: technology stack

Kotlin, Jetpack Compose and Material 3 are suitable for the Android client. AppViewModel and StateFlow organise screen state and user events; Repository handles HTTP and approved snapshots through OkHttp and coroutines. DataStore suits small account-specific bookmark/preferences data. Keystore-backed SessionStore protects credentials, while ReminderStore and ReminderReceiver handle local alarms and notifications. This state-driven UI structure follows Android's Compose architecture guidance [2].

A single Python FastAPI application with Pydantic validation and SQLAlchemy persistence keeps the backend manageable. FastAPI provides request validation and OpenAPI documentation [3]. SQLAlchemy supports the transactions needed to publish approved changes and audit entries together [4]. These capabilities suit the project; implementation and testing still determine correctness.

PostgreSQL supports the deployed shared database, while SQLite keeps isolated development and testing simple. Compose pins PostgreSQL 17. The same-origin staff portal can use plain HTML, CSS and JavaScript without a separate frontend framework or npm build. Authentication responsibilities include Argon2 password hashing, account roles, bounded sessions, email verification/reset links and server-side identity verification.

Separate adapters keep external services independent of the UI and domain validation. The current integration uses Google Maps for display, Google Routes for journeys and OneMap for address lookup. The SRS still names OneMap for routing, so that difference needs to be reconciled in the specification. Docker/Compose and a maintenance process cover deployment, expiry and backups. The scope does not justify a message broker or Kubernetes.

3.3.2 - Comparison with the earlier recommendation

The recommendations are mostly the same. The recorded earlier response [1] already recommends Kotlin/Compose, one FastAPI application, local bookmarks, transactions and a small external adapter. It suggests SQLite for the course prototype and explicitly leaves deployment concurrency for later review. PostgreSQL deployment is therefore an extension of that advice rather than a wholesale reversal.

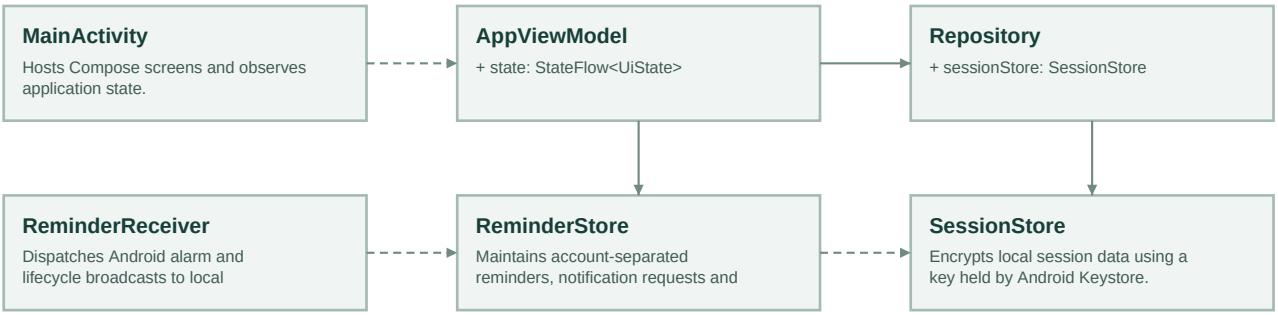
The important changes are registered accounts/sessions and email flows, account-separated reminders, the browser staff portal, and the Google map/route integration. The updated account and reminder relationships justify credential storage, lifecycle and scheduling responsibilities. They do not require changing the client language or backend framework. Google routing follows the implemented integration decision, not an inference forced by the class diagram.

No major language/framework preference changed, so the conditional follow-up in 3.3.2(2) is not applicable. The core stack remains suitable. Input scope and project constraints changed, while model identity and the complete old diagram are unavailable; differences cannot be attributed solely to a diagram change or model variability.

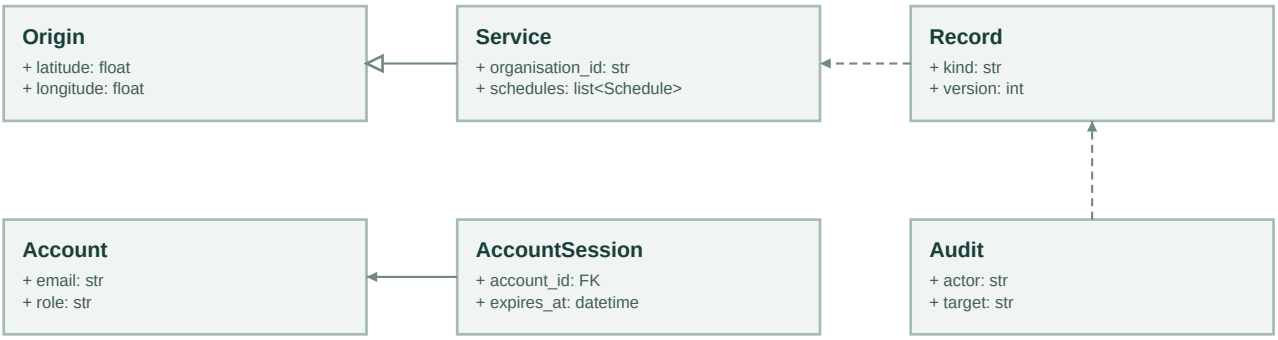
Updated class model

The two Jarbobo class diagrams below separate Android coordination from backend validation and persistence. They show selected classes; the accompanying source map covers the remaining concepts. The PDF adapts the saved Jarbobo specifications for print rather than reproducing an editor screenshot.

JARBOBO / ANDROID COORDINATION



JARBOBO / BACKEND VALIDATION AND PERSISTENCE



Solid: association. Hollow triangle: inheritance. Dashed: dependency or logical JSON/ID mapping.

Print layout from saved Jarbobo class specs. Source links and relation details remain in the editable files.

How the diagram maps to source

Android classes are in android/app/src/main/java/sg/foodbridge/navigator/. MainActivity and AppViewModel connect Compose screens to Repository, SessionStore and ReminderStore. ReminderReceiver is an Android BroadcastReceiver. Android Service is a JSONObject wrapper in Repository.kt.

Backend Service, Origin, Search, Schedule and Observation are Pydantic models in backend/app/domain.py. Record, Account, AccountSession, AccountLink and Audit are SQLAlchemy models in backend/app/db.py. AccountSession and AccountLink declare account foreign keys. Record-to-domain and Audit-to-target links are logical JSON/identifier relationships, not SQL foreign keys.

Provider and Report schemas are also in domain.py. Services refer to organisations and optionally providers; reports can target either a service or a provider. The editable Jarbobo files link the displayed classes to source. The fuller Mermaid model and architecture notes cover the omitted relationships, function-based controllers, adapters and local preference concepts.

3.3.3 - TODO-only project template

Text prompt B

Using the updated system description and class diagram on pages 1 and 3, generate a minimal folder hierarchy for FoodBridge Navigator in Kotlin/Jetpack Compose, Python/FastAPI and plain HTML/CSS/JavaScript. Mirror the existing repository's module boundaries and filenames, including accounts, reminders, approved-data cache, moderation/audit, external adapters, maintenance and tests. Do not invent a class for every conceptual entity.

Every file must contain only a TODO comment in its language's comment syntax. Do not include imports, package declarations, classes, functions, pass statements, configuration values, markup, scripts, credentials, data, binaries or implementation code. Keep the output separate from the full application under foodbridge-ai-todo-skeleton/. Return the folder tree, a brief textual explanation and a ZIP containing only this comment-only hierarchy. It is an intentionally non-runnable template. Do not modify or relocate the full application's files.

AI response: project template

The template mirrors the implemented Android, API and browser boundaries. Accounts, reminder delivery, validation/persistence, adapters, operations and acceptance checks have explicit placeholders. Each file contains one TODO comment, including configuration and HTML/XML files. No implementation or buildable configuration is included. The archive has a single clearly named root and excludes this report, real seed data, credentials, assets and build output.

```
foodbridge-ai-todo-skeleton/
  README.md, .gitignore, .dockerignore, compose.yaml
  android/
    build.gradle.kts, settings.gradle.kts, gradle.properties
    app/build.gradle.kts, app/proguard-rules.pro
    app/src/main/AndroidManifest.xml
    app/src/main/assets/README.md, res/values/styles.xml
    app/src/main/java/sg/foodbridge/navigator/
      MainActivity.kt, AppViewModel.kt, Repository.kt
      SessionStore.kt, Reminders.kt, AccountScreens.kt
      FoodMap.kt, SearchSuggestions.kt, ServiceDetails.kt
      Providers.kt, UiComponents.kt
  backend/
    requirements.txt, Dockerfile
    app/__init__.py, main.py, domain.py, db.py, auth.py
    app/onemap.py, google_routes.py, maintenance.py, scheduler.py
    app/manage_admin.py, seed.py, retire_demo.py
    seeds/README.md
    tests/verify_backend.py, verify_staff_portal.py
  staff-portal/
    index.html, portal.css, portal.js
    account/index.html, account.js
  tools/
    start-local.ps1, setup-android.ps1, build-android.ps1
    import-reviewed-seed.ps1, import-foodconnect.ps1
  docs/{architecture,setup,operations,verification}/README.md
```

3.3.4 - Critical evaluation

I agree with retaining the three established application roots. It makes responsibilities recognisable without imposing microservices or a second frontend framework. Grouping classes into the existing cohesive modules is more faithful than creating an empty file for every SRS noun: a bookmark is local preference data; a recommendation is a result structure; a moderation item is a typed Record. The expanded architecture notes map these concepts explicitly.

The template covers the obvious structural responsibilities: sign-in/profile UI, protected local sessions, bookmarks/cache, reminder receiver, provider/details/maps, validation, persistence, audits/moderation, routing/geocoding, scheduled maintenance, tests and reviewed-input placeholders. It deliberately omits font/image binaries, actual datasets, credentials and downloaded toolchains. Those cannot be represented as comment-only implementation files. The Android asset folder has a TODO Markdown placeholder instead of invalid commented JSON.

It did not add implementation beyond the request. All template files contain one TODO comment; Python placeholders have no executable AST nodes. HTML/XML use comments only, Kotlin/JavaScript/Gradle use line comments, and shell/Python/YAML/text configuration use hash comments. The prose explanation and report stay outside the archive. The ZIP is distinguishable from the full application by its name and single foodbridge-ai-todo-skeleton root.

The template is intentionally not runnable or buildable. A new project still needs dependency declarations, manifest elements, database setup, implementation, assets and meaningful tests. The folders alone cannot establish security, accessibility, performance or correct moderation. Because this assessment uses an existing design, it can favour familiar choices; the rationale matters more than simply matching the current files.

Existing SRS differences retained for review

The supplied SRS names OneMap for routes; the active handler uses Google Routes. Backend ranking contains diet match count, distance, operating state, verification time and ID, but mandatory AND matching makes the selected-diet count constant, leaving distance as the primary varying key. Direct staff approve-and-publish edits coexist with the pending-change flow. Embedded availability observations do not have the observation ID described in the SRS dictionary. These are documentation findings; no behaviour was changed to reconcile them.

3.3.5 - Submission contents and validation

The submission consists of this report and the separate foodbridge-ai-todo-skeleton.zip. The ZIP contains 54 comment-only files under one clearly named root. Validation checks the comments, archive parity, source references and document links. The six-page PDF is rendered and reviewed for layout. Editable report and diagram sources are kept with the lab materials. Application behaviour is outside the scope of this document update.

Earlier exercise evidence and references

Recorded Lab 2 AI response

The archived Lab 2 exercise [1] records the following AI response. It provides the baseline for the comparison in section 3.3.2.

I recommend a small Android client and one server application. Build the Android UI with Kotlin and Jetpack Compose. The client can keep bookmark IDs and optional preferences on the device, while obtaining current approved service details from the server; avoid storing the user's device location after the search session.

Build a single Python FastAPI service for search, service details, public correction reports, administrator authentication and moderation. Keep matching, freshness labels, imports and approval decisions in this service so the same rules apply to every client. Use SQLite for a course prototype database containing approved organisations/services, availability observations, reports, change requests, decisions and import provenance. Use transactions for an approval and its audit record. If the team later needs deployment with more concurrent writers, reassess the database before production.

Use OneMap through a small adapter for address lookup and routes, with a timeout and an explicit unavailable state. Import only authorised partner exports or a manually verified seed dataset. Keep the first version to one mobile app, one API service and one database. Test the main search, stale-data, report-validation and moderation paths before adding extra infrastructure.

Sources

[1] Repository: reference/stitch_foodbridge_navigator/SC2006 Labs.docx, section "FoodBridge Navigator - AI technology stack exercise" / "Technology Stack Discussion". Prompt and response recovered from document XML; duplicate copies agree. The updated SRS was supplied by the user in this conversation, with use cases dated 17 October 2026.

[2] Android Developers, Compose UI architecture. https://developer.android.com/develop/ui/compose/architecture (accessed 18 October 2026).

[3] FastAPI, Features. https://fastapi.tiangolo.com/features/ (accessed 18 October 2026).

[4] SQLAlchemy 2.0, Session API. https://docs.sqlalchemy.org/en/20/orm/session_api.html (accessed 18 October 2026).

Implementation evidence: Android Kotlin sources; backend/app/main.py, domain.py, db.py, auth.py, onemap.py, google_routes.py, maintenance.py and scheduler.py; staff-portal/; backend/requirements.txt; Android Gradle files; compose.yaml. See architecture.md for the concept-by-concept mapping. Framework sources support capabilities only, not claims of production performance, security or complete SRS compliance.